

MaxAir Technical – Application Programming Interface (API)

A set of APIs are available to facilitate communication between MaxAir and other application software. An example could be integration with Amazon's Echo to enable voice control of MaxAir functions.

In order that the APIs can be called without the need for the .php file extension, the Apache2 mod_rewrite module needs to be enabled. A script file 'enable_rewrite.sh' is provided to enable this feature and can be executed using the command 'sudo bash /var/www/api/enable_rewrite.sh'.

The API's available are:

- binarySet
- boostSet
- getController
- getTemperature
- getZoneStatus
- modeSet
- getZoneData
- getFrostStatus
- getRelayState
- getLiveTemp
- awaySet
- summerSet

The APIs are used by calling using the MaxAir systems IP address and a parameter representing a Zone Name e.g. ***<http://192.168.0.100/api/binarySet?zonename=Central%20Heating>***. All APIs are capable of returning data in 'JSON' format, additionally binarySet and boostSet API can pass actions to the MaxAir system.

binarySet API

Used to return the current state of a zone or set a new state for the zone.

Example 1 - get the current state of the 'Central Heating' zone:

http://192.168.0.100/api/binarySet?zonename=Central%20Heating	<pre>{"success":true,"state":false,"state_str":"off"}</pre>
---	---

The returned JSON contains 3 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the zone's current state, 'state_str' – OFF or ON depending on the 'state'.

If a zone contains multiple controllers e.g. a Lamp zone with x individual light switches, then an array will be returned, with each element structured as above.

Example 2 - set the state of the 'Central Heating' zone to 'ON':

http://192.168.0.100/api/binarySet?zonename=Central%20Heating&state=1	<pre>{"success":true,"state":true}</pre>
---	--

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the zone's new state.

boostSet API

Used to return the current boost status of a zone or set a new boost state for the zone.

Example 1 - get the current boost status of the 'Central Heating' zone:

http://192.168.0.100/api/boostSet?zonename=Central%20Heating	<pre>{"success":true,"state":false,"state_str":"off"}</pre>
---	---

The returned JSON contains 3 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the zone's current boost status, 'state str' – OFF or ON depending on the 'state'.

If a zone contains multiple controllers e.g. a Lamp zone with x individual light switches, then an array will be returned, with each element structured as above.

Example 2 - set the boost state of the 'Central Heating' zone to 'ON':

http://192.168.0.100/api/boostSet?zonename=Central%20Heating&state=1	<pre>{"success":true,"state":true}</pre>
---	--

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the zone's new boost state.

getController API

Used to return the current ON/OFF state of the system controller.

Example :

http://192.168.0.100/api/getcontroller	<pre>{"success":true,"state":"OFF"}</pre>
---	---

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – the current ON/OFF state of the system controller.

getTemperature API

Used to return the current temperature value from a temperature sensor, the response depends on whether or not the temperature is mains or battery powered.

Example 1 - get the current temperature from the mains powered 'Hot Water' temperature sensor:

http://192.168.0.100/api/getTemperature?sensorname=Hot%20Water	<pre>{"success":true,"state":"29.40","datetime":"2021-03-30 18:03:31"}</pre>
---	--

The returned JSON contains 3 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – the current sensor temperature value, 'datetime' the time the sensor was last updated.

Example 2 - get the current temperature from the battery powered 'Conservatory' temperature sensor:

http://192.168.0.100/api/getTemperature?sensorname=Conservatory	<pre>{"success":true,"state":"23.70","datetime":"2021-03-30 18:16:52","bat_voltage":"4.01","bat_level":"85.00"}</pre>
---	---

The returned JSON contains 5 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – the current sensor temperature value, 'datetime' the time the sensor was last updated, 'bat_voltage' – current battery voltage, 'bat_level' – current battery level.

getZoneStatus API

Used to return the current status from a zone, the response depends on whether or not the temperature sensor attached to the zone is mains or battery powered.

Example 1 - get the current status from the 'Hot Water' zone, which uses a mains powered temperature sensor:

http://192.168.0.100/api/getZoneStatus?zonename=Hot%20Water	<pre>{"success":true,"status":"0","temp":"29.5","datetime":"2021-03-30 14:39:15"}</pre>
---	---

The returned JSON contains 4 components, 'success' – either true or false depending on if the API call was successful or not, 'status' – the current ON/OFF status of the zone, 'temp' – the temperature returned by the temperature sensor attached to the zone, 'datetime' the time the temperature sensor was last updated.

Example 2 - get the current status from the 'Central Heating' zone, which uses a battery powered temperature sensor:

http://192.168.0.100/api/getZoneStatus?zonename=Central%20Heatingr	<pre>{"success":true,"status":"0","temp":"29.5","datetime":"2021-03-30 14:39:15","bat_voltage":"4.01","bat_level":"85.00"}</pre>
---	--

The returned JSON contains 6 components, 'success' – either true or false depending on if the API call was successful or not, 'status' – the current ON/OFF status of the zone, 'temp' – the temperature returned by the temperature sensor attached to the zone, 'datetime' the time the temperature sensor was last updated, 'bat_voltage' – current battery voltage, 'bat_level' – current battery level.

modeSet API

Used to return the current operating 'Mode' of the system or set a new operating 'Mode'.

Example 1 - get the current operating 'Mode':

http://192.168.0.100/api/modeSet	<code>{"success":true,"mode":"TIMER"}</code>
---	--

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'mode' – the current operating 'Mode' of the system.

Example 2 - set the operating 'Mode' to 'HW':

http://192.168.0.100/api/modeSet?mode=hw	<code>{"success":true,"mode":"hw"}</code>
---	---

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'mode' – the new operating 'Mode' of the system.

Note: The parameter passed to the API can be either numeric 0 – 4 (represent the mode) or a character string "off, timer, ce, hw or both" and can be either lower or upper case.

getZoneData API

Used to return the running schedule temperature, the linked sensor temperature, the datetime of the last temperature reading, the minim and maximum temperatures for the current day and the temperature change over the last 1 hour, for either a single named zone or for all zones.

Example 1 - get the current temperature data for the Ground Floor zone.:

http://192.168.0.100/api/getZoneData? zonename=Ground%20Floor	<code>{"success":true,"id":"42","name":"Ground Floor","temp_target":"21.0","temp_actual ":"14.72","temp_datetime":"2026-04-15 07:34:05","min":"14.62","max":"14.81","d elta":"0.13"}</code>
--	---

The returned JSON contains 9 components, 'success' – either true or false depending on if the API call was successful or not, 'status' – the Zone ID, the Zone Name, the running schedule target temperature (or 0.0 if no schedule is active) and the last temperature reading for the sensor attached to the zone (if multiple sensors are attached, the average value will be returned), the datetime of the last temperature reading, the minimum temperature for the current day, the maximum temperature for the current day and the temperature change over the last 1 hour.

Example 2 - get the current temperature data for all zones.:

http://192.168.0.100/api/getZoneData	<pre>{ "success":true,"zones":[{"id":"42","name":"Ground Floor","temp_target":"21.0","temp_actual":"14.72","temp_datetime":"2026-04-15 07:32:18","min":"14.62","max":"14.81","delta":"0.13"}, {"id":"43","name":"Bedrooms","temp_target":"18.0","temp_actual":"14.93","temp_datetime":"2026-04-15 07:32:18","min":"14.87","max":"15.00","delta":"0.06"}, {"id":"44","name":"Hot Water","temp_target":"44.9","temp_actual":"14.56","temp_datetime":"2026-04-15 07:32:18","min":"14.43","max":"14.56","delta":"0.06"}]}</pre>
---	---

The returned JSON contains 6 components per zone, 'success' – either true or false depending on if the API call was successful or not, 'zones' the Zone ID, the Zone Name, the running schedule target temperature (or 0.0 if no schedule is active), the last temperature reading for the sensor attached to the zone (if multiple sensors are attached, the average value will be returned), the minimum temperature for the current day, the maximum temperature for the current day and the temperature change over the last 1 hour.

getFrostStatus API

Used to return an indication as to whether or not Frost Control is active .

Example 1 – Frost Control NOT active:

http://192.168.0.100/api/getFrostStatus	<pre>{ "success":true,"state":"Frost Control NOT Active."}</pre>
---	--

Example 2 – Frost Control active:

http://192.168.0.100/api/getFrostStatus	<pre>{ "success":true,"state":"Frost Control Active."}</pre>
---	--

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – the current state of Frost Control.

getRelayState API

Used to return the state of any relay by name.

Example 1:

http://192.168.0.100/api/getRelayState?relayname=Downstairs	<pre>{ "success":true,"state":"ON"}</pre>
---	---

LiveTempSet API

Used to return the zone ID and name to which the Live Temperature control is attached together with the current active state and target temperature setting.

Used to set the Live Temperature Control

Example 1 - get the current Live Temperature Control active state and target temperature setting:

http://192.168.0.100/api/LiveTempSet	<pre>{"success":true,"status":[{"id":"39","name":"Ground Floor","active":"0","targettemp":"25.0"}]}</pre>
---	---

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – the current active state and target temperature setting for the Live Temperature control.

Example 2 – set the active state to ACTIVE and target temperature to 25C:

http://192.168.0.100/api/LiveTempSet?active=1&targettemp=25	<pre>{"success":true,"active":"1","targettemp":25}</pre>
---	--

The returned JSON contains 3 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – the current ON/OFF state of the named relay.

awaySet API

Used to return or set the current state of the AWAY function.

Example 1 - get the current AWAY state:

http://192.168.0.100/api/awaySet	<pre>{"success":true,"state":false,"state_str":"off"}</pre>
---	---

The returned JSON contains 3 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the current state, 'state str' – OFF or ON depending on the 'state'.

Example 2 - set the state of the AWAY function to 'OFF':

http://192.168.0.100/api/awaySet?state=0	<pre>{"success":true,"state":false}</pre>
---	---

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the AWAY function new state.

summerSet API

Used to return or set the current state of the Summer Mode function.

Example 1 - get the current Summer Mode state:

http://192.168.0.100/api/summerSet	<pre>{"success":true,"state":false,"state_str": "off"}</pre>
---	--

The returned JSON contains 3 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the current state, 'state str' – OFF or ON depending on the 'state'.

Example 2 - set the state of the Summer Mode function to 'OFF':

http://192.168.0.100/api/summerSet?state=0	<pre>{"success":true,"state":false}</pre>
---	---

The returned JSON contains 2 components, 'success' – either true or false depending on if the API call was successful or not, 'state' – either true or false depending on the Summer Mode new state.